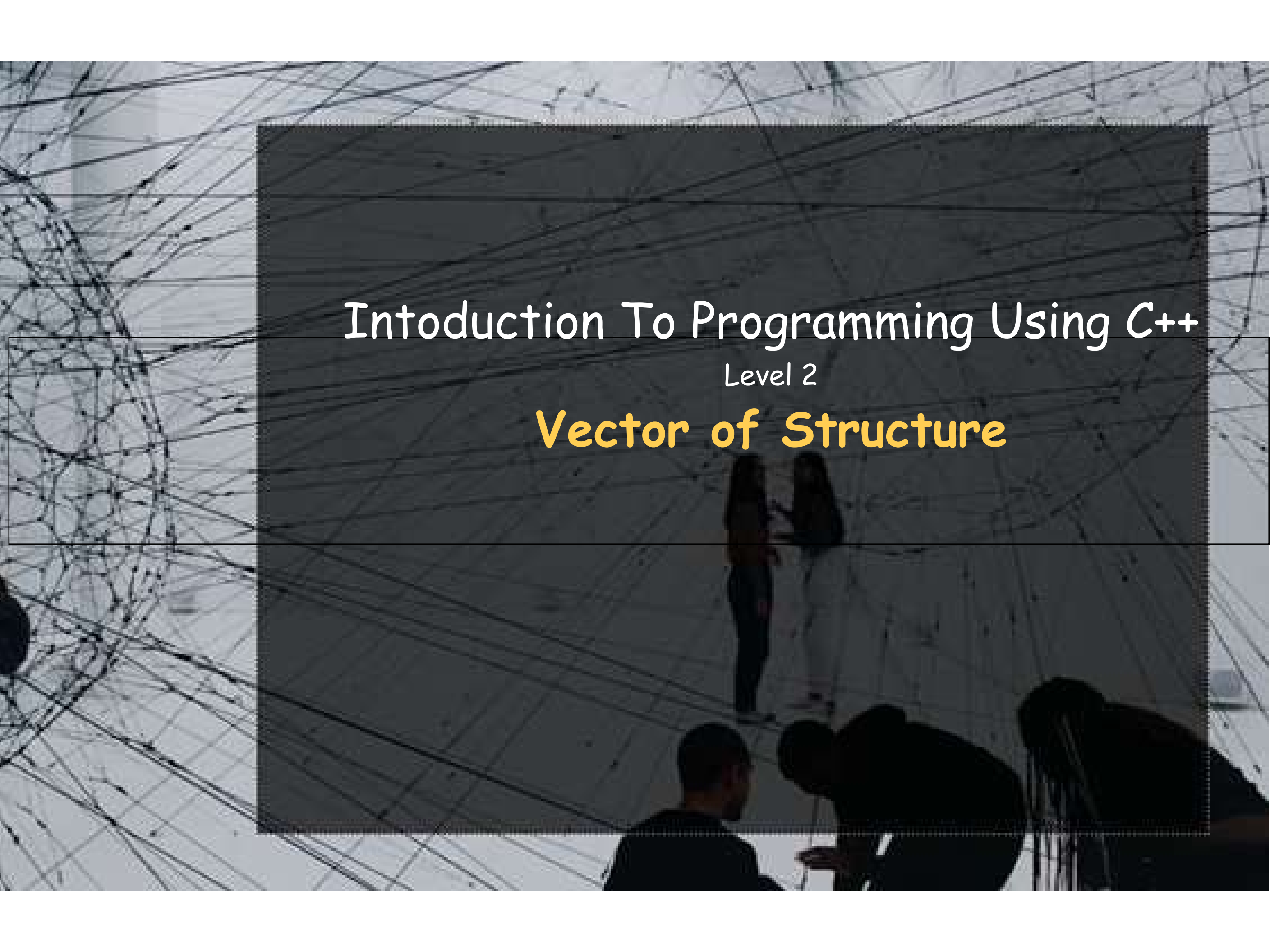


Introduction To Programming Using C++ Level 2

Vector of Structure



Introduction To Programming Using C++

Level 2

Vector of Structure

Definition

- ◆ A Vector can store elements of any valid C++ type, including user-defined `struct` types.
- ◆ This allows us to store multiple related pieces of data together inside each Vector element.
- ◆ For example, an Employee can have a `FirstName`, `LastName`, and `Salary`, and each Employee can be stored as one element in the Vector.

Explanation

◆ Defining a Structure

```
struct stEmployee  
{  
    string FirstName;  
    string LastName;  
    float Salary;  
};
```

◆ Creating a Vector of Structures

```
vector<stEmployee> vEmployees;
```

◆ Each element of `vEmployees` is an `stEmployee` object containing all three fields.

Procedures

1. Define the ``struct``.
2. Create a Vector using the Structure type.
3. Create a temporary Structure object.
4. Assign values to its members.
5. Use ``push_back()`` to add the Structure object to the Vector.
6. Repeat the process for additional elements.
7. Use a Ranged ``for`` Loop to process the stored Structures.

💡 Example

```
struct stEmployee
{
    string FirstName;
    string LastName;
    float Salary;
};

int main()
{
    vector<stEmployee> vEmployees;
    stEmployee tempEmployee;
    tempEmployee.FirstName = "Ahmad";
    tempEmployee.LastName = "Aljasem";
    tempEmployee.Salary = 5000;
    vEmployees.push_back(tempEmployee);
    tempEmployee.FirstName = "Sara";
    tempEmployee.LastName = "Alterkawi";
    tempEmployee.Salary = 300;
    vEmployees.push_back(tempEmployee);
    tempEmployee.FirstName = "Aya";
    tempEmployee.LastName = "Alterkoui";
    tempEmployee.Salary = 1000;
    vEmployees.push_back(tempEmployee);
}
```

💡 Example

```
cout << "Employees Vector:\n\n";
for (stEmployee &Employee : vEmployees)
{
    cout << "FirstName: " << Employee.FirstName << endl;
    cout << "LastName : " << Employee.LastName << endl;
    cout << "Salary   : " << Employee.Salary << endl;
    cout << endl;
}
return 0;
}
```

➡ Result

```
Employees Vector:
FirstName: Ahmad
LastName : Aljasem
Salary   : 5000
FirstName: Sara
LastName : Alterkawi
Salary   : 300
FirstName: Aya
LastName : Alterkou
Salary   : 1000
```


Note

- ◆ A Vector can store Structures just like it can store simple types such as `int`, `float`, or `string`.
- ◆ The type inside the angle brackets determines what each Vector element contains:
 `vector<int> vNumbers;`
 `vector<string> vNames;`
 `vector<stEmployee> vEmployees;`
- ◆ In the last example, every element of `vEmployees` is an `stEmployee`.

⊘ Warning

◆ Do not automatically use a non-`const` Reference everywhere.

◆ Use:

`const vector<stEmployee> &vEmployees`

◆ when the function only needs to read the Vector.

◆ Use:

`vector<stEmployee> &vEmployees`

◆ when the function intentionally needs to modify the original Vector.

◆ Passing by Reference avoids copying the Vector object itself, which is especially useful when the Vector contains many elements.

Comparison

Array

- ✓ Built-in Arrays have a fixed size.
- ✓ The size is normally specified when the Array is declared.
- ✓ When passed to a function, an Array parameter commonly behaves like a pointer to its first element.

Vector

- ✓ A Vector has a dynamic size.
- ✓ It can grow or shrink as elements are added or removed.
- ✓ Passing a Vector by value creates a copy.
- ✓ Passing it by Reference avoids copying the Vector itself.

✦ Summary

- ✓ A Vector can store Structures as its elements.
- ✓ ``vector<stEmployee> vEmployees;`` creates a Vector of ``stEmployee`` objects.
- ✓ Each Vector element can contain multiple related fields.
- ✓ ``push_back()`` adds a new Structure object to the Vector.
- ✓ The same temporary Structure object can be reused to create multiple Vector elements.
- ✓ Passing a Vector by value creates a copy.
- ✓ Passing a Vector by Reference avoids copying the Vector itself.
- ✓ Use ``const`` Reference when you only need to read the Vector.
- ✓ Use a non-``const`` Reference when you intentionally need to modify the original Vector.

Thank you 😊